



Universidad Nacional Autónoma de México

Facultad de Ciencias

# Tarea 02

*Máquinas de turing*

| No. de cuenta | Nombre                     |
|---------------|----------------------------|
| 320148204     | González Lucero Alan Uriel |
| 321225087     | Rivera Jiménez Isaac       |
| 321306102     | Tapia Sánchez Jesús        |



Complejidad Computacional

Semestre 2027-1

## Índice

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>Ejercicio 0</b>            | <b>2</b> |
| 1.1      | Respuestas . . . . .          | 2        |
| 1.2      | 1 . . . . .                   | 2        |
| 1.3      | 2 . . . . .                   | 2        |
| 1.4      | 3 . . . . .                   | 2        |
| <b>2</b> | <b>Ejercicio 1</b>            | <b>3</b> |
| 2.1      | Solución . . . . .            | 3        |
| 2.2      | 1 . . . . .                   | 3        |
| 2.3      | Tiempo de ejecución . . . . . | 3        |
| 2.4      | Ejemplo . . . . .             | 3        |
| 2.5      | 3 . . . . .                   | 4        |
| 2.6      | Tiempo de ejecución . . . . . | 5        |
| 2.7      | Ejemplo . . . . .             | 5        |
| <b>3</b> | <b>Ejercicio 2</b>            | <b>6</b> |
| 3.1      | Solución . . . . .            | 6        |
| <b>4</b> | <b>Ejercicio 3</b>            | <b>7</b> |
| 4.1      | Solución . . . . .            | 7        |
| <b>5</b> | <b>Ejercicio 4</b>            | <b>7</b> |
| 5.1      | Solución . . . . .            | 8        |

## Ejercicio 0

### Enunciado

Considerando la definición formal vista en clase de la máquina de Turing, responde las siguientes preguntas (justifica ampliamente).

1. ¿Una máquina de Turing podría escribir un símbolo en blanco en la cinta?
2. ¿El alfabeto de la cinta  $\Gamma$  puede ser igual al alfabeto de entrada  $\Sigma$ ?
3. ¿Podemos tener una máquina con un solo estado?

### Respuestas

#### 1

Si, pues el símbolo blanco  $\sqcup \in \Gamma$ . Por lo que la función de transición:

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{\leftarrow, \rightarrow\}$$

puede indicar explícitamente que se reemplazar cualquier carácter leído en la celda por un  $\sqcup$ , lo que equivale a borrar un símbolo durante el proceso de cómputo.

#### 2

Sea  $\Sigma = \{0, 1\}$  el alfabeto de entrada y sea  $\Gamma = \{0, 1, \sqcup\}$ . Consideremos que la entrada  $x$  que recibirá la máquina, es una codificación en binario. Como los elementos de la entrada  $x$ , están escritos en la cinta, es claro que  $\Sigma \subset \Gamma$ . Pero  $\Gamma \not\subseteq \Sigma$ , pues  $\sqcup \notin \Sigma$ .

$\therefore \Gamma \neq \Sigma$  no pueden ser iguales. Además, dependiendo el problema a resolver, la cinta puede requerir símbolos adicionales para el procesamiento, que no forman parte de la entrada original.

#### 3

Es posible definirlo, pero no sería útil. Pues para que una máquina de Turing realice un cómputo útil o decida un lenguaje, requiere de al menos un estado de rechazo  $q_r$  o de aceptación  $q_a$ , si la máquina consta del estado inicial:  $q_0$ :

- Si  $q_0$  se define como estado de parada, la máquina se detiene instantáneamente en el paso 0 para cualquier cadena, sin procesar la entrada.
- Si  $q_0$  no es un estado de parada, la máquina nunca podrá cambiar de estado para detenerse, entrando en un bucle infinito para cualquier entrada.

$\therefore$  Una máquina de Turing de un solo estado no puede decidir ningún lenguaje no trivial ni calcular funciones útiles. Por eso en complejidad computacional, se requieren máquinas con al menos dos estados.

## Ejercicio 1

### Enunciado

Da una descripción general de máquinas de Turing que deciden los siguientes lenguajes y determina el tiempo de ejecución de dichas máquinas.

1.  $\{w \in \{a, b\}^* \mid w \text{ contiene un número igual de } a\text{'s y } b\text{'s}\}$
  2. \*Opcional\*  $\{w \in \{0, 1, \#\}^* \mid w \text{ contiene el doble de } 0\text{'s que } 1\text{'s}\}$
  3.  $\{w \in \{a, b, c\}^* \mid w \text{ el número de } a\text{'s en } w > \text{número de } b \text{ en } w \geq \text{número de } c \text{ en } w\}$
- Ejemplifica la ejecución (o idea general) de las máquinas propuestas.

### Solución

#### 1

1. A la entrada  $x$  se le debe de anexar un símbolo inicial  $*$  y final  $+$ , teniendo ahora  $x' = *x + \mid x \in \{a, b\}^*$ . (Aclarando que  $\sqcup$  será ignorado en a la hora de buscar).
2.  $M$  lee el símbolo inicial  $*$  y se desplaza a la derecha en búsqueda de una  $a$  o  $b$ . Si se llega inmediatamente a  $+$  acepta.
3. Si  $M$  lee  $a$  imprime  $\sqcup$  y procede a buscar  $b$ . Si  $M$  lee  $b$  imprime  $\sqcup$  y procede a buscar  $a$ . Si se llega a  $+$  rechaza.
4. Cuando haya encontrado el símbolo buscado, imprime  $\sqcup$  y se desplaza a la izquierda hasta leer  $*$
5. Repetir la sucesión de pasos desde 2, hasta rechazar o aceptar.

### Tiempo de ejecución

En cada ciclo completo se eliminan exactamente dos caracteres (una  $a$ , y una  $b$ ). Por lo que se realizará un máximo de  $n/2$  ciclos antes de terminar.

En el peor de los casos, se debe recorrer toda la cinta hacia la derecha para encontrar la pareja y luego regresar al inicio. Lo que toma un tiempo proporcional a la longitud de la cadena.  $O(n)$ .

$$\therefore n/2 \times O(n) = O(n^2)$$

### Ejemplo

Sea la entrada  $x = baba$ . Aplicando el primer paso obtenemos la cinta inicial  $x' = *baba+$ .

1. **Inicio:**  $*baba+$
2. **Iteración 1:** Se lee la primera  $b$ , se borra y se busca su pareja  $a$  a la derecha.  
Estado de la cinta tras borrar el primer par  $(b, a)$ :  $* \sqcup \sqcup ba+$   
La cabeza rebobina a la izquierda hasta  $*$ .

3. **Iteración 2:** Se avanza omitiendo los  $\sqcup$ , se lee la siguiente  $b$ , se borra y se busca su pareja  $a$ .  
Estado de la cinta tras borrar el segundo par  $(b, a)$ :  $* \sqcup \sqcup \sqcup \sqcup +$   
La cabeza regresa a la izquierda hasta  $*$ .
4. **Fin:** Se avanza desde  $*$ , se omiten los  $\sqcup$  y se lee el delimitador  $+$ . La máquina **acepta**.

### 3

#### Aclaración general sobre el mecanismo de búsqueda de $M$ :

- **Salto de blancos:** La acción de "*buscar*" un símbolo significa que  $M$  se desplaza leyendo la cinta e ignorando las celdas con el símbolo  $\sqcup$ .
  - **Inercia de movimiento (caso general):** Al leer y borrar un símbolo,  $M$  continuará buscando en la **\*\*misma dirección\*\*** en la que se venía desplazando. Si alcanza el delimitador del extremo ( $+$  o  $*$ ) sin hallar el objetivo, la máquina invertirá su sentido para realizar un barrido completo hacia el extremo opuesto.
  - **Búsqueda desde un extremo (caso optimizado):** Si el símbolo es borrado justamente en una celda adyacente a un delimitador ( $*$  o  $+$ ), la máquina no necesita intentar seguir hacia esa pared; en su lugar, se reorienta inmediatamente para realizar un único recorrido directo de  $* \rightarrow +$  (o  $+ \rightarrow *$ ).
1. A la entrada  $x$  se le anexan los delimitadores de inicio y fin, teniendo  $x' = *x+$  con  $x \in \{a, b, c\}^*$ .
  2. **Fase 1 (Eliminar tríos  $c, b, a$ ):**  $M$  busca una  $c$ . Si la encuentra, la reemplaza por  $\sqcup$  y procede a buscar una  $b$ :
    - Si encuentra  $b$ , la reemplaza por  $\sqcup$  y busca una  $a$ .
    - Si encuentra  $a$ , la reemplaza por  $\sqcup$  y **regresa al inicio de la Fase 1**.
    - Si en alguno de los pasos falta la  $b$  o la  $a$ ,  $M$  **rechaza**.
  3. **Fase 2 (Eliminar pares  $b, a$ ):** Cuando ya no queden  $c$ 's en la cinta,  $M$  busca una  $b$ . Si la encuentra, la reemplaza por  $\sqcup$  y busca una  $a$ :
    - Si encuentra  $a$ , la reemplaza por  $\sqcup$  y **regresa al inicio de la Fase 2**.
    - Si no encuentra  $a$ ,  $M$  **rechaza** (pues  $\#a \leq \#b$ ).
  4. **Fase 3 (Verificación de  $a$  sobrante):** Cuando ya no queden ni  $c$ 's ni  $b$ 's en la cinta,  $M$  busca una  $a$ :
    - Si encuentra al menos una  $a$ ,  $M$  **acepta** (garantizando  $\#a > \#b \geq \#c$ ).
    - Si no encuentra ninguna  $a$ ,  $M$  **rechaza**.

## Tiempo de ejecución

Sea  $n$  la longitud de la cadena de entrada  $x$ .

- **Número de iteraciones:** Cada ciclo de la Fase 1 cancela un trío  $(c, b, a)$ , la Fase 2 cancela un par  $(b, a)$ , y la Fase 3 realiza una verificación final. Por lo tanto, se realizan a lo sumo  $O(n)$  iteraciones de borrado.
- **Costo por iteración:** En el peor de los casos, la cabeza de  $M$  realiza barridos completos a lo largo de la cinta de longitud  $n + 2$  para localizar cada símbolo o verificar los límites  $(*$  y  $+$ ). Cada barrido toma tiempo  $O(n)$ .

$$\text{Tiempo Total} = O(n) \times O(n) = O(n^2)$$

$\therefore$  El tiempo de ejecución de la máquina de Turing  $M$  en el peor de los casos es cuadrático,  $O(n^2)$ .

## Ejemplo

Sea la cadena de entrada  $w = \text{bcaaaabc}$  con  $n = 8$ , donde  $\#a = 4$ ,  $\#b = 2$ ,  $\#c = 2$ . La cinta inicial es  $x' = *\text{bcaaaabc}+$ .

### 1. Fase 1 (Cancelación de tríos $c, b, a$ ):

- **Ciclo 1:** Se localiza la primera  $c$ , la primera  $b$  y la primera  $a$ , reemplazándolas por  $\sqcup$ .  
Estado de la cinta:  $* \sqcup \sqcup \sqcup \text{aaabc} +$
- **Ciclo 2:** Se localiza la segunda  $c$ , la  $b$  restante y otra  $a$ , reemplazándolas por  $\sqcup$ .  
Estado de la cinta:  $* \sqcup \sqcup \sqcup \sqcup \text{aa} \sqcup \sqcup +$
- **Fin de Fase 1:**  $M$  realiza un barrido completo y no encuentra más símbolos  $c$ .

### 2. Fase 2 (Cancelación de pares $b, a$ ):

- $M$  realiza un barrido en la cinta buscando el símbolo  $b$ . Dado que no existen más  $b$ 's, avanza directamente a la Fase 3.

### 3. Fase 3 (Verificación de $a$ sobrante):

- $M$  busca el símbolo  $a$  y localiza al menos una de las dos  $a$ 's restantes en la cinta.
- Al encontrarla, confirma que  $\#a > \#b \geq \#c$  ( $4 > 2 \geq 2$ ) y **acepta**.

## Ejercicio 2

### Enunciado

Para cada uno de los lenguajes, da la descripción completa de una máquina de Turing (incluyendo estados, alfabetos y función de transición) que acepte  $L$ , da la función de complejidad de tiempo y ejemplifica la ejecución de la máquina propuesta mostrando las configuraciones que se producen con los ejemplares mencionados.

1. **a** Sea  $L = \{0^n 1^m 0^{n+m} \mid n, m \geq 1\}$ .

Ejemplares:

(a) 011000

(b) 010

2. **b** Sea  $L = \{a \# b \mid a, b \text{ codifican números enteros positivos en binarios y } \max(a, b) = a\}$ .

Ejemplares:

(a)  $[7] \# [5] \rightarrow 111 \# 101$

(b)  $[2] \# [4] \rightarrow 10 \# 100$

(c)  $[1] \# [1] \rightarrow 1 \# 1$

3. **OPCIONAL** Sea  $L = \{0^n 1^{n^2} \mid n \in \mathbb{N}, n \geq 1\}$

Ejemplares:

(a) 001111

(b) 000111111

### Solución

## Ejercicio 3

### Enunciado

Considera la Máquina de Turing definida por:

$$M = (\{q_0, q_1, q_2, q_a, q_e\}, \{0, 1\}, \{0, 1, \sqcup\}, \delta, q_0, q_a, q_r)$$

Describe el Lenguaje  $L_M$  para cada una de las siguientes definiciones de  $\delta$ . Para cada inciso.

· Justifica tu respuesta agregando un par de ejecuciones (al menos una de aceptación y una de rechazo) para cada inciso.

· Da la función  $f_M(b)$  del tiempo de ejecución de la máquina.

1. A.

- (a)  $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
- (b)  $\delta(q_1, 1) = (q_2, 0, \leftarrow)$
- (c)  $\delta(q_2, 1) = (q_0, 1, \rightarrow)$
- (d)  $\delta(q_1, \sqcup) = (q_a, \sqcup, \rightarrow)$

2. B.

- (a)  $\delta(q_0, 0) = (q_1, 1, \rightarrow)$
- (b)  $\delta(q_1, 1) = (q_0, 0, \rightarrow)$
- (c)  $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$

3. C.

- (a)  $\delta(q_0, 0) = (q_0, \sqcup, \rightarrow)$
- (b)  $\delta(q_0, 1) = (q_1, \sqcup, \rightarrow)$
- (c)  $\delta(q_1, 1) = (q_1, \sqcup, \rightarrow)$
- (d)  $\delta(q_1, \sqcup) = (q_a, \sqcup, \leftarrow)$

Considera que las reglas (transiciones) que no se indican de manera explícita llevan al estado de rechazo.

## Solución

## Ejercicio 4

### Enunciado

Investiga qué son las funciones de tiempo "construibles" (Time constructible functions).

1. Elige dos de las siguientes funciones y demuestra que son de tiempo construibles:

- (a)  $2n \log n$
- (b)  $3n^2$
- (c)  $3^n$

2. Da otros ejemplos (diferentes a los del inciso anterior) de funciones de tiempo construible.

3. Da un par de ejemplos de funciones que no son de tiempo construible. Justifica por qué no lo son.

## Solución

Las funciones de tiempo construibles se refiere a una función  $f(n)$  tal que existe una máquina de Turing determinista de múltiples cintas que se detiene en  $f(n)$  cuando se le da una palabra de entrada de longitud  $n$ . Dichas funciones desempeñan un papel importante en la teoría de la computación. Son interesantes por sí mismas como ejemplos de funciones muy ‘honestas’. Por lo tanto, es deseable contar con técnicas potentes para demostrar la construibilidad en tiempo de las funciones.

Por máquina de Turing nos referimos a una máquina de Turing determinista de múltiples cintas. Se supone que todas las cintas son cintas de trabajo (es decir, no utilizamos cintas especiales de entrada o salida) y que las cintas son infinitas en ambas direcciones. Generalmente  $\mathbb{N}$  denota el conjunto de todos los enteros no negativos. Se dice que una función de  $d$  variables  $f(n_1, \dots, n_d)$  de  $\mathbb{N}^d$  a  $\mathbb{N}$  es *construible en tiempo* (*ct* para abreviar) si existe una máquina de Turing con al menos  $d$  cintas que se detiene en el tiempo  $f(n_1, \dots, n_d)$  cuando se inicia con la siguiente configuración en el tiempo 0: (1) para cada  $i$  ( $1 \leq i \leq d$ ), el contenido de la  $i$ -ésima cinta es  $\dots BB1^{n_i}BB\dots$  y el cabezal escanea el 1 situado más a la izquierda (si  $n_i > 0$ ) ( $B$  denota el símbolo blanco y  $1^{n_i}$  denota la secuencia de  $n_i$  unos), y (2) el contenido de las otras cintas, si las hay, está completamente en blanco.[1]

- Elige dos de las siguientes funciones y demuestra que son de tiempo construibles:
  - a.  $2n \log n$
  - b.  $3n^2$
  - c.  $3^n$

Tomando el **inciso b)**:

Sea  $3n^2$  decimos es una función en tiempo construible si existe una máquina MT de múltiples cintas, con la entrada  $1^{n_i}$  (representación unaria de  $n$ ), produce la representación binaria de  $f(n)$  en  $O(f(n))$  pasos. Se puede simular  $M$  de la siguiente forma.

Se necesita escanear la cinta de entrada que contiene  $1^n$  para contar el número de caracteres, convirtiéndolo en un entero binario  $n$  en una segunda cinta. Después se tiene que calcular  $n^2$ . Para esto el entero binario  $n$  por sí mismo utilizando un algoritmo estándar de multiplicación binaria para obtener  $n^2$ . Multiplicar  $n^2$  por 3 (lo cual equivale simplemente a desplazar  $n^2$  un bit a la izquierda para obtener  $2n^2$  y sumarlo a  $n^2$ , o sea  $3n^2 = 2n^2 + n^2$ ).

Convertir  $1^n$  a binario requiere exactamente  $n$  pasos. La representación binaria de  $n$  tiene  $\lfloor \log_2 n \rfloor + 1$  bits. Multiplicar dos números de  $m$  bits requiere  $O(m^2)$  pasos. si tenemos,  $m = O(\log n)$ , elevar  $n$  al cuadrado requiere  $O(\log^2 n)$  pasos. Multiplicar por

3 conlleva un tiempo lineal respecto al número de bits de  $n^2$ , es decir,  $O(\log n)$  pasos.

Sumando estos valores, el tiempo total  $T(n)$  necesario para calcular la salida binaria es:  $T(n) = O(n) + O(\log^2 n) + O(\log n) = O(n)$ . Dado que  $O(n) \leq O(3n^2)$  para todo  $n \geq 1$ , así  $3n^2$  es construible en tiempo.

Tomando el **inciso c)** consideramos:

Sea  $3^n$  decimos es una función en tiempo construible si existe una máquina MT de múltiples cintas, con la entrada  $1^{n_i}$  (representación unaria de  $n$ ), produce la representación binaria de  $f(n)$  en  $O(f(n))$  pasos. Se puede simular  $M$  de la siguiente forma.

Escribir el número binario  $1(3^0)$  en la Cinta 2. Mantener la entrada  $1^n$  en la Cinta 1 para que actúe como contador del bucle. Recorrer la entrada de izquierda a derecha, y por cada 1 leído, multiplicar por 3 el numeral de la zona de trabajo. Al encontrar el primer  $\sqcup$ , detenerse.

Multiplicar por 3 en binario es una sola operación compuesta ( $3x = 2x + x$ ). Es decir, que se debe de recorrer un lugar y sumar el original. Después de tachar todos los  $n$  símbolos de la Cinta 1, la Cinta 2 contiene la representación binaria exacta de  $3^n$ .

Ahora podemos analizar el número total de operaciones realizadas por  $M$ , para la Longitud en bits tras  $i$  iteraciones, el valor en la Cinta 2 es  $3^i$ , por lo que el número de bits necesario para almacenar  $3^i$  es  $\lfloor \log_2(3^i) \rfloor + 1 \approx i \cdot \log_2(3) \approx 1,58i$ .

En la  $i$ -ésima iteración, copiar, desplazar y sumar un número binario de longitud  $O(i)$  requiere  $O(i)$  pasos. Sumar los pasos de todas las  $n$  iteraciones da una serie aritmética:  $\sum_{i=1}^n O(i) = O\left(\frac{n(n+1)}{2}\right) = O(n^2)$ .

Dado que  $O(n^2) \leq O(3^n)$  para todo  $n \geq 1$ , la máquina de Turing calcula  $3^n$ . Por lo tanto,  $3^n$  es una función construible en tiempo.

- Da otros ejemplos (diferentes a los del inciso anterior) de funciones de tiempo construible.

Muchas funciones simples como  $n^2$ ,  $n^3$ ,  $2^n$ ,  $n!$  son ct. Las demostraciones no son fundamentalmente difíciles, pero sí tediosas.

Para  $n^3$ , sucede un caso similar a  $n^2$ . Suponemos a  $M$  determinista de 3 cintas de tra-

bajo, donde cada una contiene inicialmente la entrada unaria  $1^n$ . La máquina ejecuta un esquema de tres bucles anidados. La cinta 1 actúa como contador externo y realiza  $n$  iteraciones. Para la cinta 2 realiza un recorrido completo de  $n$  posiciones por cada celda procesada en la cinta 1 ( $n^2$  iteraciones en total). Así para la cinta 3 ejecuta un barrido completo de  $n$  celdas ( $n$  pasos) por cada movimiento de la Cinta 2.

El número total de transiciones ejecutadas hasta la parada es  $T(n) = n \cdot n \cdot n = n^3$ . La máquina se detiene en exactamente  $n^3$  pasos,  $f(n) = n^3$  es construible en tiempo.

Para  $f(n) = n!$ , se puede suponer una máquina  $M$  que utiliza la cadena de entrada  $1^n$  para controlar las llamadas anidadas de  $n$  niveles de profundidad.

En la  $m$ -ésima iteración del bucle, multiplicamos  $(m-1)!$  por  $m$ . El número de bits necesarios para representar  $(m-1)!$  es  $O(m \log m)$ , y el número de bits para  $m$  es  $O(\log m)$ , multiplicarlos sería  $O((m \log m) \cdot \log m) = O(m \log^2 m)$  pasos.

Sumamos los costos  $T(n) = \sum_{m=1}^n O(m \log^2 m) = O(n^2 \log^2 n)$ , entonces  $f(n) = n!$  es construible en tiempo.[2]

- Da un par de ejemplos de funciones que no son de tiempo construible. Justifica por qué no lo son.

**La función**  $f(n) = \log n$

Para cualquier función construible en tiempo bajo la definición estándar, la máquina de Turing debe al menos leer la cadena de entrada  $1^n$  de longitud  $n$ .

Supongamos que la cadena de entrada en la cinta de la máquina de Turing es  $1^{1000}$ , de acuerdo con la función  $f(n) = \log_2 n$ , la máquina debería finalizar su cómputo y detenerse aproximadamente:

$$f(1000) = \log_2(1000) = 9 \text{ pasos}$$

Para que la máquina de Turing pueda procesar o reconocer que el tamaño de la entrada es  $n = 1000$ , el cabezal debe desplazarse celda por celda a lo largo de la cinta, por lo que leerla requiere un mínimo indispensable de 1000 pasos, solo que al llegar al paso 10, el cabezal solo ha logrado avanzar sobre los primeros 10 símbolos ( $1^{10}$ ). Quedan aún 990 símbolos de la entrada sin leer.

Leer una entrada de longitud  $n$  requiere un mínimo de  $n$  pasos físicos de desplazamiento del cabezal, como  $\log n = o(n)$ , la función crece demasiado lento: la máquina agota el presupuesto de tiempo permitido ( $\log n$  pasos) antes de haber podido leer la entrada

completa.

**Teorema de Jerarquía de Tiempo.** Aquí fungen funciones construidas mediante saltos condicionales basados en el problema de la parada o usando la diagonalización del Teorema de Jerarquía de Tiempo

Para entender por qué no es construible en tiempo, debemos examinar cómo está construida mediante una paradoja intencionada (autorreferencia y diagonalización).

Sea  $M_1, M_2, M_3, \dots$  una enumeración de todas las máquinas de Turing posibles. Definimos artificialmente una función  $T(n)$  paso a paso para cada tamaño de entrada  $n$ . Tomamos la  $n$ -ésima máquina de Turing,  $M_n$ , y le damos la entrada  $n$ .

Observamos el comportamiento de  $M_n(n)$ , por ejemplo, definir  $f(n)$  de modo que valga  $2n$  si una máquina  $M_n$  se detiene en su propia entrada, y  $2n + 1$  si no lo hace (o variaciones similares).

No es constructible porque si pudieras construir esta función eficientemente en tiempo  $O(f(n))$ , podrías usar ese algoritmo para resolver o decidir problemas indecidibles (como el problema de la parada), lo cual genera una contradicción.[3]

## Referencias

- [1] Kobayashi, K. (1985). On proving time constructibility of functions. *Theoretical Computer Science*, 35, 215–225. Disponible en: [https://doi.org/10.1016/0304-3975\(85\)90015-5](https://doi.org/10.1016/0304-3975(85)90015-5).
- [2] Blais, E. (2026). *CS 365: Models of Computation – Time Complexity* [Notas de clase]. David R. Cheriton School of Computer Science, University of Waterloo. Disponible en: <https://cs.uwaterloo.ca/~eblais/cs365/w26/time>.
- [3] Mathematics Stack Exchange. (2014). *Why are the hierarchy theorem proofs called diagonalization?* Disponible en: <https://math.stackexchange.com/questions/658854/why-are-the-hierarchy-theorem-proofs-called-diagonalization>.